Experiment 4: Finite-State Machine for Morphological Parsing in Python

1. Aim

To write a Python program to implement a finite-state machine for morphological parsing to generate plural forms of English nouns.

2. Procedure

1. Define a morphological parsing function that accepts a singular English noun.
2. Check if the noun ends with sibilant suffixes (`'s'`, `'x'`, `'z'`, `'ch'`, `'sh'`) and append `'es'`.
3. Check if the noun ends with a consonant followed by `'y'` and replace `'y'` with `'ies'`.
4. Append `'s'` for all other regular nouns.
5. Display the singular nouns and their generated plural forms.

3. Program

```
def pluralize(noun):
    if noun.endswith(('s', 'x', 'z', 'ch', 'sh')):
        return noun + 'es'
    elif noun.endswith('y') and noun[-2] not in 'aeiou':
        return noun[:-1] + 'ies'
    return noun + 's'

nouns = ["cat", "bus", "city", "box", "boy"]
for n in nouns:
    print(n, "->", pluralize(n))
```

4. Output

```
cat -> cats
bus -> buses
city -> cities
box -> boxes
boy -> boys
```

5. Result

Thus, the Python program was successfully executed to implement a finite-state machine for morphological parsing to generate plural nouns.